

enclawed: A Configurable, Sector-Neutral Hardening Framework for Single-User AI Assistant Gateways

Alfredo Metere
Enclawed LLC
alfredo.metere@enclawed.com

May 1, 2026

Abstract

We present *enclawed*, a hard-fork hardening framework built on top of the OpenClaw single-user personal artificial intelligence (AI) assistant gateway. *enclawed* targets deployments that need attestable peer trust, deny-by-default external connectivity, signed-module loading, and a tamper-evident audit trail — typically regulated industries such as financial services, healthcare, defense contracting, regulated R&D, and government enclaves. The framework ships in two flavors: an **open** flavor that preserves OpenClaw compatibility while still emitting audit, classification, and data-loss-prevention (DLP) signals, and an **enclawed** flavor that activates strict allowlists, Federal Information Processing Standards (FIPS) cryptographic-module assertion, mandatory module-manifest signature verification, and high-assurance peer attestation for the Model Context Protocol (MCP). The classification ladder is fully data-driven: a deploying organization selects from five built-in presets (generic, US-government, healthcare, financial services, three-tier) or supplies its own JSON. We accompany the implementation with a security review, a 356-case test suite (261 unit tests, 95 adversarial pen-tests) covering tamper detection, signature forgery, fetch- and raw-socket egress bypass, audit-log truncation and re-ordering, trust-root mutation, DLP evasion, prompt injection, code injection, and biconditional admission for net-capable extensions; real-time human-in-the-loop control (per-agent pause / resume / stop and approval queues); a memory-bounded secure transaction buffer with rollback (default cap 50% of system RAM, configurable); a strict-mode TypeScript type-check of every framework file; and a GitHub Actions workflow ready for continuous integration. A newly added `.mjs` primitive — the biconditional extension-admission gate (`extension-admission.mjs`) — extends the paper’s skill trust schema to non-skill extensions, so every loadable extension declaring `net.egress` must carry a signed manifest, an explicit per-extension host allowlist, and (in the enclawed flavor) a verification level $\geq$ `tested`. The four-level verification lattice is now closed at the top: a companion stack of four `.mjs` primitives (`skill-formal-static.mjs`, `skill-formal-types.mjs`, `skill-formal-bmc.mjs`, `skill-formal-bundle.mjs`) plus a command-line interface (CLI) at `scripts/skills-formal-verify.mjs` produces a signed proof-carrying bundle the runtime can re-check at load time, raising a skill from `tested` to `formal` via composable static effect-containment, refinement-typed dispatch, and bounded model checking against the runtime biconditional. The unit-test inventory correspondingly grows from 208 to 261 unit tests; together with the 95 adversarial pen-tests this is a 356-case suite. **enclawed is a hardening framework, not an accredited compliance certification.** The deploying organization remains responsible for hardware, validated cryptographic modules, certified facilities, and assessor sign-off.

1 Introduction

Generative AI has crossed quickly from prototype into the operational fabric of regulated industries. Financial-services firms use large language models (LLMs) for research summarization

touching material non-public information (MNPI); healthcare systems apply them to protected health information (PHI); defense contractors handle controlled unclassified information (CUI) and International Traffic in Arms Regulations (ITAR)-controlled materials; legal teams rely on them for privileged-counsel work; pharmaceutical R&D operates on embargoed clinical-trial data. Each of these settings is governed by frameworks that pre-date generative AI but apply to it directly — the Health Insurance Portability and Accountability Act (HIPAA) [8], the European Union General Data Protection Regulation (GDPR) [7], the Payment Card Industry Data Security Standard (PCI DSS) [9], the International Organization for Standardization / International Electrotechnical Commission (ISO/IEC) 27001 [5], the American Institute of Certified Public Accountants Service Organization Control 2 (SOC 2) trust-services criteria [6], the National Institute of Standards and Technology (NIST) Special Publication 800-53 [1], and NIST 800-171 [4] — and increasingly by AI-specific regimes such as the European Union Artificial Intelligence Act (EU AI Act) [28], the NIST AI Risk Management Framework (NIST AI RMF) [26], and ISO/IEC 42001 [27] governing artificial-intelligence management systems.

Two observations motivate this work. First, the dominant open-source multi-channel AI gateways were designed for the consumer “personal-assistant” market and ship with cloud-first defaults that are fundamentally incompatible with regulated-enclave deployment. Second, the security failure modes of generative AI are no longer hypothetical: documented incidents include training-data extraction at scale [31, 32], indirect prompt injection through retrieved content [20, 21, 22], the 2023 Samsung corpus leak through interactive cloud application programming interfaces (APIs), and the 2024 *Moffatt v. Air Canada* decision [35] establishing operator liability for AI assistant outputs. The Open Web Application Security Project (OWASP) Top 10 for LLM Applications [29] and MITRE Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS) [30] now codify the relevant attack surface in industry-recognized form.

1.1 The regulated-enterprise AI gap

Personal AI gateways such as OpenClaw [25] exemplify the problem. The upstream project’s positioning is explicit: a single-user assistant that “answers you on the channels you already use,” bundling 24+ chat platforms (WhatsApp, Telegram, Slack, Discord, Signal, iMessage, Matrix, Microsoft Teams, and more) and 40+ cloud LLM providers (OpenAI, Anthropic, Google, Mistral, Groq, AWS Bedrock, Azure OpenAI, OpenRouter, Fireworks, etc.) by default. Its plugin model encourages community-published modules installed via `npm` with no enforced signature verification. Its policy posture is permissive: external channels and providers are enabled the moment credentials are configured. Audit, classification, and DLP are not part of the framework’s contract.

For a casual community user — one whose worst-case agent action is to leak a chat transcript or run up an API bill — this is a defensible design. Even there, however, the same defaults are not benign: a permissively-configured OpenClaw deployment that wires an agent to a robot, a home-automation bus, a 3D printer, a CNC mill, an electric door, a vehicle controller, an irrigation valve, or any other real-world actuator can quietly turn a chat-level mistake into property damage, data loss, or physical harm. Prompt-injection, hallucinated tool calls, third-party module compromise, and unbounded egress are not academic risks once the loop reaches a physical device. So the “community user” boundary is itself thinner than the upstream defaults assume.

For a financial-services trading floor handling MNPI, a hospital system processing PHI, a defense contractor with CUI access, or any organization subject to ISO 27001 / SOC 2 / HIPAA / PCI / Federal Risk and Authorization Management Program (FedRAMP) / U.S. Department of Defense (DoD) / U.S. Department of Energy (DoE) controls, every one of those defaults becomes an explicit policy violation:

- *External egress to consumer messaging platforms* violates boundary protection (NIST

800-53 SC-7 [1]) and information-flow control (AC-4).

- *Cloud LLM API usage* routes regulated content through third parties whose data-processing terms typically permit retention and, until contractually disclaimed, training reuse — in direct conflict with HIPAA business-associate constraints [8, §164.308(b)], GDPR Article 28 processor obligations [7], and most data-residency policies.
- *Unsigned community plugins* expose the gateway to supply-chain compromise — the threat the NIST Secure Software Development Framework (SSDF) [33] addresses through SI-7 / SR-3 / SR-4, and which the U.S. Cybersecurity and Infrastructure Security Agency (CISA) / National Security Agency (NSA) / Federal Bureau of Investigation (FBI) joint guidance on AI deployment [34] explicitly calls out as a top operational risk.
- *Permissive defaults with no audit chain* make incident reconstruction impossible, conflicting with NIST 800-53 AU-2/3/9/10 and HIPAA §164.312(b) audit-control requirements.
- *No classification labels* means an output may carry PHI / MNPI / CUI to a recipient cleared for none of it, with the host silent at every step.

1.2 Why a configuration-only fix is insufficient

A natural response is “just disable the cloud channels and providers, add a logging plugin, and require approved modules.” We argue this is not enough, for three structural reasons.

Defaults are policy. A platform whose out-of-the-box state contains 78 forbidden modules invites accidental enablement at every upgrade, every onboarding, every plugin install. Removing the modules from the source tree — not just from the configuration — is the only durable defense, because it makes the unsafe state *unreachable* rather than merely *unselected*. Compliance posture must be a property of the binary, not a property of a configuration file an operator can revert.

The plugin-loading path is the trust boundary. OpenClaw’s plugin loader does not verify signatures, does not consult any trust root, and does not reason about clearance. Adding “a logging plugin” is itself a plugin — it inherits the same unsigned, unverified loading path. The trust boundary must be *below* the plugin layer, enforced by the host before any plugin code runs, and locked against post-boot mutation. Retrofitting this into a project whose contract assumes ambient plugin trust is a re-architecture, not a configuration.

Upstream cannot adopt these constraints. OpenClaw’s product identity — a multi-channel personal assistant — is incompatible with deny-by-default external connectivity, FIPS-required boot, mandatory signature verification, and attested peer trust. Even if upstream maintainers were willing to accept patches that imposed those defaults, doing so would break their published contract with thousands of community deployments. Forking is therefore not merely convenient but structurally necessary: the regulated-enclave product must diverge from the consumer-assistant product at the binary level. We say explicitly that upstream OpenClaw *will not* adopt these constraints, not because the upstream maintainers are inattentive to security — they are not — but because the constraints are incompatible with the product they ship.

1.3 Three design commitments

This paper describes *enclaved*, a hard fork of OpenClaw rebuilt around three design commitments that address the gap above:

1. **Always-on policy.** The classification framework activates unconditionally at process start, before any plugin or transit module is imported, so downstream code observes a

guarded environment from the first instruction. There is no “compatibility mode” that disables the framework in production.

2. **Two flavors.** An **open** flavor preserves OpenClaw behavior with the framework running in warn-only mode for development and non-regulated deployment; an **enclaved** flavor enforces strict deny-by-default policy, FIPS assertion, mandatory module-manifest signing, and attested peer trust. Selection is by a single environment variable read once at boot and then immutable for the process lifetime.
3. **Data-driven classification.** The level ladder is not hardcoded; the deploying organization selects from five built-in presets (sector-neutral generic, U.S.-government, healthcare-HIPAA, financial-services, three-tier) or supplies its own JavaScript Object Notation (JSON) document. The framework is sector-neutral by default — a U.S.-government preset and DoE Q-clearance and L-clearance templates exist as opt-ins, not as the canonical defaults.

The framework is sector-neutral by default. Built-in presets cover financial services (MNPI / privileged-counsel), healthcare (PHI / sensitive PHI), a generic three-tier scheme, and full US-government markings, with the canonical defaults industry-generic (`PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED < RESTRICTED-PLUS`).

1.4 Comparison with the broader landscape

OpenClaw is one of several open-source projects that mediate between a human user and one or more language models. Adjacent offerings include LibreChat [43], Open WebUI [44], AnythingLLM [45], Jan.ai [46], Lobe Chat [47] — multi-provider chat front-ends — plus agent-orchestration frameworks such as AutoGen [48] and the LangChain ecosystem [49]. A separate cluster of AI-security tools targets narrower slices of the problem: NeMo Guardrails [50] and Llama Guard [51] focus on content moderation at the model boundary; Lakera Guard [52] addresses prompt injection as a hosted scanning service.

We surveyed each of these against the security properties enclaved is designed to deliver, summarized in Table 1. The honest picture is that no other publicly-available offering, as documented at the time of writing, combines mandatory module signing, a classification lattice with a configurable scheme, tamper-evident audit, real-time human-in-the-loop control, a memory-bounded rollback buffer, and attested peer trust into a single cohesive framework that runs entirely on locally-hosted inference. The closest competitors each address one or two of these properties:

- Local-LLM front-ends (Open WebUI, AnythingLLM, Jan.ai) deliver point (1) — they avoid cloud egress by binding to local runtimes — but ship without policy, classification, audit, or signed-module enforcement. Their plugin or knowledge-base ingest paths are unverified, which is the same supply-chain gap noted in §1.1 for OpenClaw.
- Multi-provider gateways (LibreChat, Lobe Chat) prioritize reach over restriction; cloud providers are first-class citizens and removal requires user effort that is not enforced. Their plugin / preset distribution is community-driven without cryptographic attestation.
- Agent frameworks (AutoGen, LangChain ecosystem) are libraries rather than gateways, and the security posture is whatever the embedding application provides. They have no notion of deployment-flavor strictness, no built-in egress allowlist, and no rollback on partial-failure of an agent’s actions.
- Content-guardrail systems (NeMo Guardrails, Llama Guard) address output filtering — a complement to, not a replacement for, classification-driven flow control. They do not

enforce information-flow rules, do not produce a tamper-evident audit trail, and do not gate module loading.

- SaaS prompt-injection scanners (Lakera Guard) by design route content through an external service, which is incompatible with the no-egress requirement of regulated enclaves.

Where a competitor genuinely addresses one of enclaved’s properties, enclaved adopts the same primitive rather than reinventing it: the DLP scanner and prompt-shield (§4.5, §8) are deliberately a thin keyword-and-regex layer because content-classification depth is the job of dedicated tools like Llama Guard, which a deploying organization can wire in addition. enclaved’s contribution is the host-level chassis that any of these complementary tools can sit inside, with the chassis itself taking responsibility for the properties the tools cannot deliver alone (signed module loading, classification lattice, deny-by-default policy, attested peer trust, HITL controls, rollback).

Project	Local-only by default	Mand. signed modules	Class. lattice	Tamper- evident audit	DLP at output	HITL stop	Tx rollback	Attested peer trust	Sector-neutral config scheme
enclaved (this work)	yes (enclaved)	yes	yes	yes	yes	yes	yes	yes (MCP)	yes
OpenClaw [25]	no	no	no	no	no	no	no	no	no
LibreChat [43]	no	no	no	no	no	no	no	no	no
Open WebUI [44]	yes	no	no	no	no	no	no	no	no
AnythingLLM [45]	opt-in	no	no	no	no	no	no	no	no
Jan.ai [46]	yes	no	no	no	no	no	no	no	no
Lobe Chat [47]	no	no	no	no	no	no	no	no	no
AutoGen [48]	N/A (library)	no	no	no	no	partial	no	no	no
LangChain [49]	N/A (library)	no	no	no	partial	partial	no	no	no
NeMo Guardrails [50]	N/A (component)	no	no	no	yes	no	no	no	no
Llama Guard [51]	N/A (model)	no	no	no	yes	no	no	no	no
Lakera Guard [52]	no (SaaS)	no	no	no	yes	no	no	no	no

Table 1: Landscape comparison. Properties documented as of the projects’ publicly-stated features at time of writing; “partial” means the property is achievable but not enforced by the project itself, and “N/A” means the project is a different kind of artifact (library, component, model) rather than a hostable gateway and the column does not directly apply. enclaved is the only entry that combines all nine properties as a host-level framework around which the others can be composed.

Non-claim. This work does not constitute a certification. Real audits — ISO 27001, SOC 2, the Health Information Trust Alliance (HITRUST) common security framework, FedRAMP, DoD/DoE high-side Authority to Operate (ATO), PCI DSS Report on Compliance (RoC) — all require accredited hardware, validated cryptographic modules (FIPS 140-3 [2] where the regime demands), certified facilities, evaluation by a qualified assessor, and management sign-off. enclaved is a code-level scaffold; the gaps a deploying organization must close are catalogued in Section 10.

Contributions.

- A two-flavor classification framework with sector-neutral defaults and built-in presets for finance, healthcare, government, R&D, and a minimal three-tier scheme.
- A user-configurable classification scheme model that accepts arbitrary level vocabularies, validated against rank-contiguity and name-uniqueness invariants.
- An Ed25519 module-signing system with a per-signer clearance-approval trust root and boot-time pre-verification.
- A hash-chained, append-only audit log with concurrent-append serialization, deep payload sanitization against log injection, and independent third-party verification.

- An adversarial test suite of 95 cases targeting tamper detection, signature forgery, fetch-and raw-socket egress bypass, audit-log truncation and re-ordering, trust-root mutation, DLP evasion, prompt injection, code injection, and biconditional admission for net-capable extensions, alongside 261 unit tests, all passing on Node 22.
- A GitHub Actions workflow that runs the suites on push, PR, and weekly cron, plus a strict-mode TypeScript typecheck of all 22 framework files.

2 Background and Threat Model

2.1 Choice of access-control model

Information security has produced a small library of formal access control models, each emphasizing a different property. We surveyed the candidates against three criteria: (i) the primary risk in our deployments is unauthorized *disclosure* of regulated data, not modification or fraud; (ii) the policy must be *mandatory* — imposed by the host, not configurable by users or modules; (iii) the ladder of sensitivity levels must be *reusable* across sectors that already think in tier vocabularies (PHI, MNPI, CUI, etc.). Table 2 summarizes the alternatives we considered.

Model	Primary property	Why not the primary fit	Use in enclaved
Bell-LaPadula [10]	confidentiality	—	primary axis
Biba [36]	integrity	does not constrain disclosure	complementary, future work
Clark-Wilson [37]	integrity via well-formed transactions	overengineered for chat-mediated AI; assumes app-defined IVPs	inspires the rollback buffer (§4.6)
Brewer-Nash [38]	dynamic conflict-of-interest separation	sector-specific (financial); orthogonal to tier flow	easily layered via compartments
Harrison-Ruzzo-Ullman (HRU) access matrix	general expressiveness	undecidable safety; no enforced lattice	rejected
Role-based access control (RBAC) [39]	role-based authorization	no notion of data sensitivity; group→resource only	rejected as primary axis
Attribute-based access control (ABAC) [40]	attribute expressions	expressive but harder to formally verify; risk of policy drift	adopted within compartments / releasability slots

Table 2: Access-control model survey. Bell-LaPadula is selected as the primary axis because the deployments enclaved targets are fundamentally confidentiality-driven (PHI, MNPI, CUI, embargoed research). Biba’s integrity dual and Clark-Wilson’s transaction discipline are complementary and inform §4.6 (rollback buffer); RBAC and ABAC are layered as compartment / releasability assignments rather than as the primary flow rule.

Concretely we chose Bell-LaPadula for five reasons:

1. *Confidentiality is the first-order risk.* Every regime in Section 10’s gap list is a confidentiality regime first (PHI under HIPAA [8], MNPI under U.S. Securities and Exchange Commission (SEC) rules, CUI under NIST 800-171 [4], classified information under NIST 800-53 [1]). Integrity and availability matter, but only after confidentiality is established.
2. *The lattice is mechanizable.* Dominance, least upper-bound, and reading/writing predicates are simple, total, decidable functions on integer ranks plus subset tests on compartment sets. They are easier to formally verify than the rule-evaluation of Clark-Wilson or the policy expressions of ABAC.

3. *It maps directly to existing vocabularies.* US-government classification, healthcare PHI tiers, and financial MNPI all naturally express as a rank ladder plus optional compartments. The same code consumes any of them via the configurable scheme of §6.
4. *It is mandatory by construction.* The lattice is a property of the host, evaluated on every label comparison. Modules cannot override it.
5. *Biba and Clark-Wilson compose orthogonally.* A future integrity layer can use Biba’s no-read-down / no-write-up on a parallel axis without modifying the existing lattice; the transaction buffer (§4.6) already implements the well-formed transaction discipline of Clark-Wilson at the action layer.

2.2 Bell-LaPadula formal model

Each subject and object carries a label $\langle \ell, C, R \rangle$ where ℓ is a numeric rank, C a set of compartments, and R a set of releasability caveats. Label a dominates label b ($a \succeq b$) iff $\ell_a \geq \ell_b$ and $C_b \subseteq C_a$. Reading is allowed when the subject dominates the object (no-read-up); writing is allowed when the object dominates the subject (no-write-down). Combination yields the least upper bound. This lattice arithmetic is implemented in `src/enclaved/classification.ts` and is independent of the level vocabulary.

2.3 Threat model

We assume a single-tenant, single-user gateway running inside a high-trust enclave. The user holds the deploying organization’s highest applicable trust tier. The system must:

1. never egress to the public Internet or any external channel/provider;
2. use only locally-hosted inference (e.g. Ollama [18], vLLM [16], LM Studio [19], SGLang [17], local NVIDIA Inference Microservice (NIM));
3. refuse to render or echo data above the user’s authorized tier;
4. refuse to write data below its origin classification (no-write-down);
5. produce a tamper-evident audit trail of every model interaction;
6. encrypt every persistent artifact at rest with a key bound to organization-controlled hardware;
7. block known sensitive-data markings, secrets, and personally identifiable information (PII) from leaving the gateway via any output channel;
8. make any deviation a hard, loud failure — not silent degradation.

We map these requirements to multiple control frameworks simultaneously (Table 3): NIST 800-53 [1], ISO/IEC 27001 [5], NIST Cybersecurity Framework (CSF) 2.0 [3], SOC 2 trust-services criteria (TSC) [6], GDPR [7], the HIPAA Security Rule [8], PCI DSS [9], and the Cybersecurity Maturity Model Certification (CMMC) layered on NIST 800-171 [4]. The deployment-side complement to this mapping — the controls the deploying organization must still deliver — is enumerated in Section 10.

Control area	enclawed surface	NIST 800-53	ISO 27001	NIST CSF	SOC 2	GDPR	HIPAA
Access enforcement	<code>classification.ts.policy.ts</code>	AC-3	A.9.4	PR.AC-4	CC5.1, CC6.1	Art. 32(1)(b)	164.312(a)(1)
Information flow	<code>egress-guard.ts</code>	AC-4, SC-7	A.13.1	PR.PT-4	CC6.6	Art. 32(1)(b)	164.312(e)(1)
Security attributes	<code>classification.ts + scheme</code>	AC-16	A.8.2	PR.DS-5	CC1.4	Art. 5(1)(f)	164.308(a)(2)
Audit logging	<code>audit-log.ts + subsystem patch</code>	AU-2/3/9/10	A.12.4	DE.CM-1, DE.AE-3	CC7.2, CC7.3	Art. 30, 32(1)(d)	164.312(b)
Crypto module	<code>crypto-fips.assertFipsMode()</code>	IA-7, SC-13	A.10.1	PR.DS-1	CC6.1	Art. 32(1)(a)	164.312(a)(2)(iv)
Encryption at rest / in transit	<code>crypto-fips.ts</code>	SC-8/13/28	A.10.1, A.13.2	PR.DS-1/2	CC6.1, CC6.7	Art. 32(1)(a)	164.312(e)(2)(ii)
Continuous monitoring	<code>audit + subsystem + egress.onDeny</code>	SI-4	A.12.4	DE.CM, DE.AE	CC7.2	Art. 32(1)(d)	164.308(a)(1)(ii)(D)
Data leakage / DLP	<code>dlp-scanner.ts</code>	SI-12	A.13.2.3	PR.DS-5	CC6.7	Art. 5(1)(f)	164.312(e)
Memory hygiene	<code>zeroize.ts</code>	MP-6, SI-16	A.8.3	PR.IP-6	CC6.5	Art. 32(1)(a)	164.310(d)(2)(i)
Least functionality	<code>host admission gate (Sec. 3.2)</code>	CM-7	A.9.1	PR.AC-3	CC6.3	Art. 25	164.312(a)(1)
Module integrity	<code>module-signing + trust-root</code>	SI-7, CM-5, SR-3/4	A.12.5, A.14.2.5	PR.IP-1, PR.DS-6	CC8.1	Art. 32(1)(b)	164.312(c)(1)

Table 3: Control mapping. Each row links a framework surface to the corresponding requirement in seven major frameworks; CMMC L2/L3 inherits NIST 800-171 [4] and PCI DSS [9] adds Req. 3, 4, 7, 10, 11 alongside NIST 800-53. The deploying organization picks the column that fits its regime. (Rendered on a dedicated landscape page; cells use natural-width 1 columns and the whole table is uniformly scaled to fill the page.)

Aspect	open (default)	enclaved
Channel/provider/host allowlists	not enforced	strict deny-by-default
Module signatures	warn-only	required, must verify
Signer-clearance approval	informational	required, exact match
FIPS assertion at boot	off (configurable)	on (default)
Trust root mutation post-boot	permitted	locked
globalThis.fetch re-assignment	permitted	frozen non-configurable
Module without manifest	loaded with warning	rejected
MCP peer attestation	verified, fail → warn	verified, fail → deny

Table 4: Flavor matrix. Both flavors run the same framework code; the distinction is enforcement strictness.

2.4 Out of scope

The framework does *not* provide accredited cryptography (FIPS 140-3 or NSA Type-1), an accredited cross-domain solution (CDS), identity binding (Common Access Card (CAC), Personal Identity Verification (PIV), Security Assertion Markup Language (SAML), or OpenID Connect (OIDC)), kernel-level mandatory access control (SELinux Multi-Level Security (MLS)), Write-Once-Read-Many (WORM) audit shipping, or facility security. These remain the deploying organization’s responsibility. Section 10 enumerates each.

3 Architecture

3.1 Two flavors

enclaved selects a flavor at boot via the `ENCLAWED_FLAVOR` environment variable. Table 4 contrasts the two.

3.2 Module set: cloud modules gated, not source-stripped

Earlier drafts of this paper argued that the safest cut was to *delete* the 78 cloud-channel and cloud-provider module directories from the upstream `extensions/` tree so the unsafe state would be `UNREACHABLE` rather than `UNSELECTED`. After porting from upstream OpenClaw, the framework now ships *all* 134 module directories (cloud channels, cloud LLM providers, external search/browser/webhook modules, and local-capable modules) and delegates rejection to the host admission gate (`src/plugins/manifest-registry.ts`). In the `enclaved` flavor, every module that does not present a signed `enclaved.module.json` verifying against the trust root is denied at load time *before any plugin code is imported*; in the `open` flavor an unsigned module loads with a warning. The unreachability property therefore shifts from a source-tree property (“the directory does not exist”) to a verified-manifest property (“the directory exists, but no signed manifest grants admission”). This shift gives operators of the open flavor access to the broader upstream module catalog while preserving the enclaved flavor’s deny-by-default posture, and it lets the framework absorb upstream extension churn without re-running deletion sweeps. As of this draft `enclaved-oss` ships 130 extension directories: 28 carry a signed `enclaved.module.json` (the local-capable inference and core extensions: Ollama, vLLM, LM Studio, SGLang, NVIDIA NIM Local, ComfyUI, the media/memory/speech cores, `openshell`, `qa-channel`, `phone-control`, `device-pair`, and `diagnostics-otel`); 96 are gated-unsigned (cloud channels and cloud LLM providers whose `enclaved.module.json` is intentionally absent

so the enclaved admission gate rejects them); 2 are utility-only directories that present no plugin metadata at all. The bundle verifier (`scripts/verify-enclaved-bundle.mjs`) reports the signed-versus- gated counts on every CI run.

3.3 Runtime singleton and bootstrap

Activation is a single call at the top of `src/entry.ts`, before any plugin or transit module is imported:

```
const { bootstrapEnclaved } = await import("./enclaved/bootstrap.js");
await bootstrapEnclaved();
```

`bootstrapEnclaved()` reads the flavor and classification scheme from the environment, loads the policy, opens the audit log, installs the egress guard (with `freeze:true` in `enclaved`), pre-verifies every module manifest under `extensions/`, locks the trust root in `enclaved`, and stores the runtime on a `Symbol.for("enclaved.runtime")` slot of `globalThis` so downstream patches can consult it without creating import cycles. The boot record (with active scheme id, flavor, FIPS state, and allowlist contents) is the genesis hash-chain entry.

4 Implementation

The framework is delivered in two parallel surfaces: a TypeScript (TS) twin under `src/enclaved/` that bundles with the upstream OpenClaw build, and a canonical reference under `enclaved/src/` written in plain Node ECMAScript Modules (`.mjs`) with zero runtime dependencies. The canonical surface runs under `node -test` without `pnpm install`, which keeps the standalone security suite cheap to exercise and trivial to audit. Table 5 summarizes file counts and lines of code per tree.

Repository layout. The framework is hosted as `enclaved-oss` — the open-source, MIT-licensed, self-contained, public-bound git repository this paper documents. It carries the OpenClaw fork base, the framework primitives (classification, policy, audit, DLP, crypto wrapper, zeroize, module signing, trust root, HITL, transaction buffer, prompt shield, two-layer egress guard, biconditional extension admission), the canonical `.mjs` reference, the standalone test suite, this paper, and *the full mirrored OpenClaw `extensions/` catalog* (~130 directories, both local-capable and cloud). Cloud extensions are admitted only when their `enclaved.module.json` is signed by a trust-root key; the unsafe state is unreachable through the admission gate, not absent from the source tree. A separate proprietary companion product, `enclaved-enclaved`, builds on this open foundation and is distributed under separate commercial terms; its internal architecture is outside the scope of this paper.

4.1 Hash-chained audit log

Every record is canonicalized as JSON with sorted keys, prepended with the previous record's SHA-256, and hashed. Three properties matter:

- *Tamper detection.* Editing a record in place breaks the chain at the next record's `prevHash` comparison, which `verifyChain()` reports as `brokenAt`.
- *Concurrent-write safety.* `append()` chains every write onto an internal `Promise` queue so two simultaneous callers cannot both consume the same `lastHash`. Without this guard, parallel emitters (e.g. the egress-guard `onDeny` callback racing with `createSubsystemLogger`) would produce a self-inconsistent chain.

Surface (tree)	Files	LOC
<i>enclaved-oss/ (open source, MIT)</i>		
TypeScript framework (<code>src/enclaved/*.ts</code> , integration shims)	32	~3,956
Canonical <code>.mjs</code> reference (<code>enclaved/src/</code>)	18	~2,286
Standalone unit + pen tests (<code>enclaved/test/</code>)	25	~3,535
Upstream patches (4 files)	4	+82
<code>src/entry.ts</code>		+10
<code>src/plugins/channel-validation.ts</code>		+27
<code>src/plugins/provider-validation.ts</code>		+27
<code>src/logging/subsystem.ts</code>		+18

Table 5: Code footprint of the open-source repository. The OSS tree is self-contained and fully buildable, testable, and publishable on its own.

- *Log-injection neutralization.* A `deepSanitize` pass strips C0 control characters from every string in `type`, `actor`, `level`, and the entire payload tree, replacing them with `U+FFFD`. This both prevents an attacker from spoofing a fake JSONL record by embedding a newline in a payload string, and ensures the on-disk bytes match the bytes that were hashed. `__proto__`, `constructor`, and `prototype` keys are filtered out before canonicalization so a payload object cannot smuggle prototype-pollution shapes into the audit hash.

4.2 Egress guard

The egress guard now ships in two layers. The first layer, `installEgressGuard()`, replaces `globalThis.fetch` with a wrapper that resolves the target hostname via the Web Hypertext Application Technology Working Group (WHATWG) URL parser (yielding case-normalized hostnames, ignoring the `Host` header, and rejecting embedded credentials) and checks against an allowlist. In the `enclaved` flavor the property is set via `Object.defineProperty` with `writable:false` and `configurable:false`, making subsequent reassignment throw under ECMAScript Modules (ESM) strict mode.

Raw-socket layer. Earlier drafts noted that the `fetch` wrapper explicitly did not protect against raw `net.Socket` use; that gap is now closed in JS by `installRawSocketGuard()`, which patches `Socket.prototype.connect`. The patch unwraps Node’s internal three argument forms (`(options, cb)`, `(port, host, cb)`, and the normalized-array form used by `net.createConnection` and `http(s).Agent.createConnection`) and applies the allowlist check to the resolved `host`. Because `tls.TLSSocket` extends `net.Socket`, and because both `http.Agent` and `https.Agent` ultimately invoke `Socket.prototype.connect`, the patch transitively covers `fetch`, `http`, `https`, `tls`, and any extension that imports `node:net` directly. In the `enclaved` flavor the patch is installed with the same `writable:false`, `configurable:false` freeze pattern used for `fetch`.

VPN-only mode. The `enclaved-flavor` policy (`defaultEnclavedPolicy`) defaults to `requireVpnGateway:true` with the RFC 1918 CIDR set (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`). The deploying organization replaces these with the exact CIDR its VPN exposes. Under VPN-only mode every IPv4 destination must either appear on the literal hostname allowlist (covering loopback, mDNS `.local`, and pre-resolved gateway names) or fall inside one of the configured CIDRs. A small zero-dependency `ipInCidr()` matcher implements the check so the canonical `.mjs` surface remains fully runtime-dependency-free. IPv6 destinations are denied unless explicitly allow-listed by literal address; deploying organizations that run a v6 VPN must add the matching CIDR via the future v6-aware matcher.

Biconditional admission for net-capable extensions. `extension-admission.mjs` extends the paper’s skill trust schema (§5) to non-skill extensions. Every loadable extension is the tuple $(M, \text{content}, \sigma)$, where the manifest M now also records a `verification` level (`unverified` / `declared` / `tested` / `formal`) and, if M declares the `net.egress` capability, an explicit `netAllowedHosts` target list. The admission gate rejects, in the `enclaved` flavor: any unsigned manifest, any manifest whose signer is absent from the trust root, any tampered manifest (the canonical bytes now cover `verification` and `netAllowedHosts` so post-signing field changes invalidate σ), and any `net.egress` declaration at `verification` below `tested`. In the `open` flavor those same conditions surface as warnings instead of denials. At runtime, `installPerExtensionEgressGuard()` narrows the deployment-wide allowlist to the per-extension declared hosts and records a `biconditional.violation` record on every $S \setminus D \neq \emptyset$ deviation (an extension reaching a host it never declared). A post-hoc check (`biconditionalNetCheck`) compares declared and observed host sets and flags the dangerous direction; $D \setminus S$ over-declaration is benign and surfaces only as advice.

Limits. Even both JS layers together do *not* cover (a) native modules that perform syscalls below `libuv`, (b) child processes spawned via `node:child_process` that invoke external binaries (`curl`, `wget`), or (c) memory-mapped or shared-memory channels inside the same machine. For a true classified enclave the deploying organization **MUST** add kernel-level egress controls (`nftables`, extended Berkeley Packet Filter (eBPF), network namespaces, optional one-way diode) on top. The JS layer is deliberately the minimum-surface-cheapest defense, sufficient to catch JS-level misbehavior by ported third-party extensions but explicitly insufficient alone for accreditation.

4.3 Closing the verification lattice: the formal-verification stack

The trust schema’s four-level lattice (`unverified` < `declared` < `tested` < `formal`) ships in the manifest, but until recently only the first three levels had a corresponding admission rule: `formal` required a mechanically-checkable proof of capability containment that the framework did not produce. The newly added stack — `skill-formal-static.mjs`, `skill-formal-types.mjs`, `skill-formal-bmc.mjs`, `skill-formal-bundle.mjs`, all under `enclaved/src/` — closes that gap with three composable methods plus a proof-carrying bundle the runtime re-checks at load time. The full development is the subject of a separate companion paper [57]; we summarise here only the parts that matter to a deploying organization asking “what does `verification: formal`” now mean operationally?”.

Method A — script-side static effect containment. `skill-formal-static.mjs` walks every script colocated with `SKILL.md` (Python, shell, Node, TypeScript) and computes a sound over-approximation of the system effects it can produce, in the parent paper’s capability vocabulary (`net.egress`, `fs.read`, `fs.write.{rev,irrev}`, `tool.invoke`, `spawn.proc`, `publish`, `pay`, `mutate.schema`). Reflective constructs (`eval`, `exec`, `dynamic import`) taint the analysis to the lattice top so the verdict is conservative by construction. The verdict is admitted only if the over-approximated effect set is contained in the manifest’s declared `caps`.

Method B — refinement-typed dispatch. `skill-formal-types.mjs` exposes `buildRefinedDispatch(M)`, which returns a frozen dispatcher that throws `RefinementError` on any envelope whose capability is outside $M.\text{caps}$. The dispatcher is the runtime’s only ingress to the host application programming interfaces (APIs); an envelope an adversarially-prompted large language model (LLM) emits cannot reach a host API without passing the type predicate. The data-processing inequality [58] bounds the per-envelope leakage at $\log_2(|D| + 1)$ bits, where $D = M.\text{caps}$.

Method C — bounded model checking. `skill-formal-bmc.mjs` runs an exhaustive depth-first search over the abstract envelope state space at bound K (default 8) and checks, on every symbolic trace, that the runtime biconditional of [10] holds (no world-state change without a corresponding admitted envelope). The state-space size is $(|D| + 1)^K$; for deployments with $|D| \leq 10$ and $K \leq 8$ the search explores at most $11^8 \approx 2.1 \times 10^8$ traces, which discharges in seconds.

Proof-carrying skill bundle. The CLI `scripts/skills-formal-verify.mjs` composes the three methods, hashes each evidence file with a sorted-key canonical encoding, and signs a four-file bundle (`static.json`, `types.proof.json`, `smt.unsat.json`, `manifest.attest.json`) with the same Ed25519 `module-signing` primitive used for the manifest itself. The runtime’s bundle re-checker (`verifyFormalBundle` in `skill-formal-bundle.mjs`) re-runs all three methods on the live source, compares hashes against the cached evidence, and verifies the attestation signature. A drift between cached and freshly-computed verdicts aborts admission with a `method-A-cache-miss` (or `method-B`, `method-C`) reason; the runtime never trusts the bundle’s producer. A worked end-to-end demonstration ships at `skills/_formal-demo/`.

The full developer-facing reference for the `SKILL.md` format — mandatory front-matter fields, the capability vocabulary, the canonicalisation and Ed25519 signing convention, the seven-step bootstrap re-check protocol, and the proof-carrying bundle layout — is published at <https://docs.openclaw.ai/tools/skill-manifest-spec> [59] and is the source of truth skill authors and runtime integrators are expected to consume.

4.4 Configurable classification scheme

A scheme is a JSON document conforming to:

```
{
  "id": "acme-2026",
  "description": "ACME Corp internal data classification policy v3.2",
  "levels": [
    { "rank": 0, "canonicalName": "Public",          "aliases": ["P"] },
    { "rank": 1, "canonicalName": "Internal",        "aliases": ["I"] },
    { "rank": 2, "canonicalName": "Customer Data",   "aliases": [] },
    { "rank": 3, "canonicalName": "Privileged",      "aliases": ["legal"] }
  ],
  "validCompartments": ["FINANCE", "ENG", "LEGAL"],
  "validReleasability": ["NDA", "EYES_ONLY"]
}
```

`parseClassificationScheme()` enforces three invariants: ranks must be contiguous starting at zero; every name (canonical and alias) must be unique across the whole scheme after case-insensitive normalization; non-empty levels list. The active scheme governs `format()`, `parse()`, `makeLabel()` range validation, manifest `clearance` validation, and required-tier checks at the MCP attestation layer. Five presets ship out of the box: `enclaved-default` (six tiers; generic + US-gov merged), `us-government` (UNCLASSIFIED to TOP SECRET//SCI), `healthcare-hipaa` (PHI / Sensitive-PHI / Research-Embargoed), `financial-services` (MNPI / Privileged-Counsel), `generic-3-tier` (Public / Internal / Restricted).

4.5 DLP scanner

A regex-based scanner with three pattern families: *sensitive-data markings* (industry banners, U.S.-government classification banners, DoE Restricted Data, Sensitive Compartmented Information (SCI) codewords, distribution caveats); *cloud and vendor secrets* (Amazon Web Services

(AWS), Google Cloud Platform (GCP) service account, Microsoft Azure storage key, GitHub, GitLab, OpenAI, Anthropic, Slack, Stripe, JSON Web Tokens (JWTs), Privacy-Enhanced Mail (PEM)-encoded private keys); and *international PII* (email address, ITU-T E.164 phone number, credit-card primary account number (PAN), International Bank Account Number (IBAN), U.S. Social Security Number (SSN)). Input length is capped at 1 MiB by default to bound CPU exposure to regular-expression denial-of-service (ReDoS)-shaped patterns; callers may pass `onOversize: 'truncate'` to scan the prefix. Severity levels (`low`, `medium`, `high`, `critical`) drive a `redact()` entry point that replaces matches above a threshold with `[REDACTED]`. The scanner is a backstop, not a primary control: regex DLP cannot detect paraphrased sensitive content, optical-character-recognition-only content, or any pattern not in its catalog.

4.6 Human-in-the-loop control

`src/enclaved/hitl.ts` provides per-agent sessions with a state machine `PENDING → RUNNING ⇌ PAUSED → {STOPPED, COMPLETED, FAILED}`, a real-time event stream on the controller (EventEmitter-based, with both a typed channel per event name and a generic `event` channel), and an approval queue. Each `AgentSession` exposes:

- `pause()`, `resume()`, `stop(reason)`, `complete()`, `fail(reason)` — explicit operator controls. `stop` is final; the state machine forbids transitions out of any terminal state.
- `checkpoint()` — a cooperative cancellation point the agent code awaits between actions. It throws `AgentStoppedError` if the session has been stopped, and blocks until `resume()` if the session is paused. We chose cooperative over preemptive cancellation deliberately: Saltzer and Schroeder’s *complete mediation* principle [41] argues for explicit checkpoints over forced thread termination, which in a single-threaded async runtime can leave shared state in an inconsistent half-mutated form.
- `proposeAction(actionType, payload)` — if the action type belongs to the session’s approval-required set, the call awaits a human decision (allow / deny) routed through the controller’s approval queue. Deny raises `ApprovalDeniedError`; if the agent is stopped while the call is awaiting the human, it instead raises `AgentStoppedError` (the approval is no longer relevant).

The `HitlController` maintains the session registry, the approval queue, and a `stopAll(reason)` mass-halt operation. Every state transition and every approval request/resolution is emitted as an event and (when an `AuditLogger` is wired) appended to the hash-chained audit log under the `hitl.<type>` namespace. A separate operator UI (CLI, web, or remote) subscribes to the event stream to render live status and to call `resolveApproval(id, decision)` when a human approves or denies a queued action. Sub-second latency is the design goal; the controller does no I/O on the hot path.

4.7 Secure transaction buffer with rollback

The framework treats every reversible action an agent takes as a *transaction*, in the sense of Gray’s classical formulation [42]. The deploying organization registers each action’s inverse at the moment of execution; the buffer (`src/enclaved/transaction-buffer.ts`) holds the resulting transaction record so it can be rolled back later if a human, an automated guardrail, or a failed downstream operation demands undo.

Memory-bounded rollback window. The buffer is sized as a configurable percentage of system RAM (default 50%, configurable via the `ramPercent` constructor option, with an absolute `maxBytes` override). The size cap matches the eviction policy of Clark-Wilson [37]’s

well-formed transaction discipline applied to a finite resource: as long as the action sits in the buffer it can be inverted; once evicted, it has been implicitly committed. A new record that would push the buffer past the cap auto-commits the oldest records FIFO until enough room is freed.

Tamper-evident chain. Each transaction is hash-chained to its predecessor with SHA-256, in the same shape as the audit log of §4.1. `verifyChain()` performs an independent walk so an operator (or a separate auditor process) can detect silent tampering of the in-memory record set. Every record, rollback, commit, and eviction is emitted to the audit log when one is wired.

Rollback semantics. `rollback(n)` pops the `n` most recent records and runs each `inverse` in LIFO order. An inverse that throws is reported in the result's `errors` array and an audit `transaction.rollback-failed` record is appended, but rollback continues so a single broken inverse cannot block restoration of the rest of the window. After rollback the chain is re-anchored to the new tail so subsequent records continue cleanly.

5 Module Signing and Trust Root

Each module ships an `enclaved.module.json` alongside its `package.json`:

```
{ "v": 1,
  "id": "ollama",
  "publisher": "enclaved bundled",
  "version": "0.1.0",
  "clearance": "internal",
  "capabilities": ["model-provider"],
  "signerKeyId": "enclaved-bundled-dev-2026",
  "signature": "<base64 Ed25519 signature>" }
```

The signature is an Ed25519 [13] signature over a stable canonical encoding of the manifest body excluding `signature` itself, so signing is idempotent. The trust root binds each public key to a list of clearance tiers it is approved to attest. Verification passes when (i) the manifest's `signerKeyId` resolves to a signer in the trust root, (ii) the signer is approved for the manifest's declared clearance, (iii) the signature verifies against the canonical bytes, and (iv) any caller-required clearance is met. In the `enclaved` flavor, the bootstrap calls `lockTrustRoot()` after the deploying organization's signers are loaded, after which any `setTrustRoot()` or `resetTrustRoot()` call throws `TrustRootLockedError`.

The module loader is two-staged: `preloadModuleDecisions(rootDir)` walks the modules directory once at boot, parses every manifest, runs `checkModule()` against the active flavor and trust root, and stashes the per-id decisions on the runtime singleton; the synchronous validation chokepoints (channel- and provider-validation, both patched into upstream OpenClaw) read the cached decisions without performing async I/O on a hot path. Modules denied at preload are rejected when their channels or providers subsequently try to register, with a corresponding `module.deny.channel` or `module.deny.provider` audit record.

6 Configurable Classification — Worked Examples

Table 6 shows the five built-in schemes side by side.

A deploying organization selects a scheme via:

Rank	default	us-government	healthcare-hipaa	financial-services	3-tier
0	PUBLIC	UNCLASSIFIED	PUBLIC	PUBLIC	PUBLIC
1	INTERNAL	CUI	INTERNAL	INTERNAL	INTERNAL
2	CONFIDENTIAL	CONFIDENTIAL	PHI	CONFIDENTIAL	RESTRICTED
3	RESTRICTED	SECRET	SENSITIVE-PHI	MNPI	—
4	RESTRICTED-PLUS	TOP SECRET	RESEARCH-EMB.	PRIVILEGED-COUNSEL	—
5	SCI	TOP SECRET//SCI	—	—	—

Table 6: Built-in classification schemes (column headers are scheme ids; rank-4 of `healthcare-hipaa` is `RESEARCH-EMBARGOED`, abbreviated for table fit). Aliases such as `q-cleared` $\equiv$ rank 4 of `us-government` are accepted by `parse()` and by manifest clearance validation.

```
ENCLAWED_FLAVOR=enclaved \
ENCLAWED_CLASSIFICATION_SCHEME=healthcare-hipaa \
ENCLAWED_AUDIT_PATH=/var/log/enclawed/audit.jsonl \
node openclaw.mjs gateway run --bind loopback
```

For a custom scheme, the same env var is set to a JSON file path; `loadSchemeByName` accepts an optional `allowedDirs` parameter that rejects paths outside a vetted configuration directory.

7 Prompt Injection Defenses

`src/enclawed/prompt-shield.ts` provides defensive helpers for text that flows from untrusted sources (channel input, retrieved document, tool output) into a model prompt. The framework does not promise complete prompt-injection defense — that is an open research problem [20, 21, 22] — but it neutralizes the most common silent-confusion vectors so they surface as visible content rather than control signals to the model.

`sanitizeForPrompt(text)` performs five operations in order:

1. `stripControlChars` replaces ASCII C0 control characters (the byte range 0x00–0x1F excluding TAB) with the Unicode replacement character U+FFFD.
2. `stripBidi` removes the Unicode bidirectional-override range U+202A–U+202E and U+2066–U+2069.
3. `stripZeroWidth` removes the zero-width characters U+200B–U+200D, U+2060, and U+FEFF.
4. `neutralizeRoleBoundaries` prefixes any line starting with `system:` / `assistant:` / `user:` / `tool:` / `function:` (with optional leading whitespace or Markdown emphasis) with a literal `[USER-CONTENT]` marker so the model sees them as quoted rather than as a fresh role boundary.
5. `neutralizeFences` inserts a zero-width character before any line consisting of triple back-ticks so the closing fence cannot break out of a quoted code block the host wraps untrusted input in.

`detectInjection(text)` returns finding identifiers without modifying the text, including a heuristic for ALL-CAPS imperative override phrases (“IGNORE ALL PREVIOUS INSTRUCTIONS”, “OVERRIDE the above messages”, etc.).

8 Evaluation

8.1 Test methodology

The framework ships with two test surfaces. The canonical `.mjs` suite uses Node’s built-in `node:test` runner [24] and depends only on `node:crypto`, allowing verification without any `npm` install. A vitest twin under `src/enclaved/integration.test.ts` exercises the same surface in the upstream OpenClaw `pnpm test` pipeline.

8.2 Unit test coverage

Table 7 enumerates the per-module unit-test counts. All 261 unit tests pass on Node 22.

8.3 Adversarial pen-test coverage

Table 8 enumerates the per-file pen-test counts. All 95 pen-tests pass on Node 22.

8.4 Bugs found and fixed during construction

The unit and pen suites caught the following real defects during development; each was fixed prior to publication:

1. `crypto-fips.deriveKey` — `ERR_CRYPTO_INVALID_SCRYPT_PARAMS` at boundary. Lifted `maxmem` explicitly to 64 MiB.
2. `classification.makeLabel` — compartments backed by `Set` were mutable through `.add()` despite `Object.freeze`. Switched to deduplicated, sorted, frozen arrays.
3. `classification.parse` — auto-promoting `TOP_SECRET` to `TOP_SECRET_SCI` on any non-releasability segment broke `format/parse` round-tripping. Removed; SCI is a separately representable head.
4. `audit-log.append` — concurrent calls raced on `lastHash`, producing a broken chain. Fixed with internal Promise queue.
5. `audit-log` log-injection — payload strings containing raw newlines could spoof a fake JSONL record. Fixed with `deepSanitize` pass before record construction.
6. `trust-root` — post-boot `setTrustRoot` could substitute attacker-controlled signers. Fixed with `lockTrustRoot()` called by the bootstrap in `enclaved`.
7. `dlp-scanner` — unbounded input could trigger pathological regex CPU. Added 1 MiB cap with `throw` or `truncate`.
8. `egress-guard` — a hostile extension could reassign the `fetch` wrapper or patch `Socket.prototype.connect` to bypass the guard. Fixed with the `freeze` option, which uses `defineProperty` (`writable:false`, `configurable:false`) on both layers, so reassignment throws under ESM strict mode.
9. Scheme parser — accepted a non-string `canonicalName` via `String(...)` coercion. Now requires a real string with `TypeError`.
10. Scheme loader — unwrapped `JSON.parse` surfaced a bare `SyntaxError`. Wrapped with file-path context; added `allowedDirs` path-traversal allowlist.

8.5 Aggregate result

All 356 tests (261 unit + 95 pen) pass on Node 22 in approximately 1.0second. Every framework TypeScript file type-checks clean under `tsc -strict -noEmit`. The continuous-integration (CI) workflow at `.github/workflows/enclaved-security.yml` runs both suites on push, pull request (PR), and weekly cron, plus a strict-mode typecheck, the bundle verifier (`scripts/verify-enclaved-bundle.mjs`), and a boot-time signature verification of every shipped module manifest. The verifier counts both signed extensions (admitted in the enclaved flavor) and gated-unsigned extensions (rejected by the host admission layer in the enclaved flavor) and exits non-zero on any signature mismatch or unreachable-tree violation.

9 Related Work

Multi-channel AI assistants. OpenClaw [25] is the upstream personal-AI gateway from which enclaved is forked. LiteLLM [19] provides a multi-provider proxy but does not address classification or attestation. Local-inference engines such as Ollama [18], vLLM [16], LM Studio [19], and SGLang [17] are the inference targets that survive the deletion catalog of Section 3.2.

Information-flow control. The Bell-LaPadula model [10] originated the no-read-up / no-write-down formulation enclaved implements. Operating-system mandatory-access-control (MAC) implementations such as SELinux [11] are the kernel-layer counterpart that enclaved assumes the deploying organization will configure separately.

Prompt-injection research. Greshake et al. [20] formalized indirect prompt injection; Perez and Ribeiro [21] catalogued early attack patterns; Schulhoff et al. [22] demonstrated competition-grade jailbreaks. enclaved’s `prompt-shield` module addresses the data-sanitization layer of this problem only; the broader research direction remains open.

Audit logs. Tamper-evident logging via hash chains is a classical technique [12, 14]. enclaved implements a single-machine variant; production deployments should layer WORM storage or off-host shipping (e.g. to a separate audit aggregator) on top.

Formal verification of skill capability containment. The companion paper *Methods for Formal Verification of Agent Skills* [57] develops the three composable methods (sound static effect containment, refinement-typed dispatch, and bounded model checking against the runtime biconditional) that this paper’s `skill-formal-*.mjs` stack implements. That paper positions the work against the broader landscape — the Model Context Protocol (MCP) and OpenAI function-calling for schema-validated tool dispatch, Open Policy Agent and AWS Cedar for runtime authorisation, Sigstore / Supply-chain Levels for Software Artefacts (SLSA) / The Update Framework (TUF) for supply-chain attestation, and OWASP’s LLM Top 10 / MITRE’s Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS) for the attack catalogues this work neutralises — and gives the soundness theorems this paper does not reproduce. The two papers are intended to be read together: this paper documents the framework that contains the proof-carrying bundle; the companion paper documents the proofs.

10 Limitations and Gaps the Deploying Organization Owns

The framework cannot, by itself, deliver an accredited deployment. The following gaps are explicitly out of scope and must be filled by the deploying organization’s security and compliance program:

- *Identity binding.* `canRead` enforces Bell-LaPadula, but no identity layer ships. Integrate the organization’s identity provider (IdP) — using SAML, OIDC, or a smart card — and bind a clearance label to every session.
- *OS-level mandatory access control.* Defense in depth requires SELinux MLS or equivalent enforcing the same lattice outside the JavaScript process.
- *Cross-trust-zone transfer.* The framework forbids egress; a real deployment needs an accredited cross-zone control (a data diode, a file-transfer appliance, a manual review queue, or a vendor CDS).
- *Validated cryptographic module.* The deployment must link against a regime-appropriate validated module (FIPS 140-3 for U.S. federal / DoD / DoE / FedRAMP; Common Criteria (CC) Evaluation Assurance Level (EAL) evaluations for some EU and Asia-Pacific (APAC) regimes).
- *Key management.* Replace passphrase parameters with HSM-backed key references (Public-Key Cryptography Standards #11 (PKCS#11), the Key Management Interoperability Protocol (KMIP), or the vendor’s software development kit (SDK)).
- *At-rest coverage.* Wrap every persistent artifact — credentials, sessions, model weights, swap, core dumps — not just the credential planner.
- *Audit log durability.* Ship records to WORM media or an isolated audit aggregator; the on-disk hash chain alone cannot detect trailing truncation.
- *Memory hygiene.* JavaScript strings cannot be zeroized; audit every secret-handling path. Disable swap and core dumps.
- *Plugin allowlist by signature, not id.* Bind each allowed plugin to a signed manifest hash and reject any drift.
- *Cascade work from gated extensions.* Cloud channels and cloud LLM providers ship in the source tree but without a signed manifest, so the host admission gate refuses to load them in the enclaved flavor. Upstream code may still carry legacy references (legacy migrations, doctor repair paths, hard-coded fallbacks) that assume those plugins exist. The deploying organization must run `pnpm tsgo && pnpm test && pnpm build` to enumerate and fix them.

11 Conclusion

enclawed packages a coherent set of hardening primitives — classification labels, deny-by-default policy, egress allowlist, hash-chained audit, DLP scanning, FIPS-gated at-rest crypto, secret zeroization, Ed25519 module signing with a clearance-aware trust root, and prompt-injection sanitization — behind a single one-call bootstrap that runs before any plugin loads. The two-flavor design preserves upstream compatibility for community deployments while supporting strict enforcement in regulated settings. The classification ladder is data-driven, accommodating any sector’s vocabulary through five built-in presets or a custom JSON scheme. The 356-test suite (261 unit + 95 adversarial) and the CI workflow let a deploying organization continuously verify that the security primitives still hold against the attack patterns enumerated in Section 9. The verification lattice is now closed at the top: the four-primitive `skill-formal-{static,types,bmc,bundle}.mjs` stack plus the `scripts/skills-formal-verify.mjs` CLI raise an admitted skill from `tested` to `formal` via composable static effect-containment, refinement-typed dispatch, and bounded model checking, and emit a signed proof-carrying bundle the runtime re-checks at every load.

The framework is not a substitute for accredited cryptography, hardware, facilities, or assessor sign-off; the gaps in Section 10 remain the deploying organization’s responsibility. What enclaved provides is an explicit, well-tested, and continuously-verifiable scaffold that the organization’s compliance program can build on rather than reinvent.

Availability and Licensing

enclaved-oss is the open-source framework documented in this paper. The framework primitives — 29 TypeScript files (including the integration shims) plus the parallel 22-file `.mjs` reference (of which the four `skill-formal-*.mjs` primitives ship as `.mjs`-only by design and are exercised end-to-end through `skill-formal-*.test.mjs`), the unit and adversarial pen-test suites, the canonical `SKILL.md` manifest specification at `docs/tools/skill-manifest-spec.md`, the `skills/_formal-demo/` worked example, the formal-verification CLI `scripts/skills-formal-verify.mjs`, the upstream OpenClaw fork base, the full mirrored OpenClaw extension catalog (~130 directories, gated by the host admission layer in the enclaved flavor), this paper, the companion formal-verification paper at `papers/skills-formal-arxiv/`, and the CI workflow — are released under the **MIT License** and hosted at <https://github.com/enclaved/enclaved> [56]. Copyright (c) 2026 Enclaved LLC; copyright (c) 2025 Peter Steinberger (upstream OpenClaw). MIT permits commercial use, modification, redistribution, and incorporation into proprietary projects, subject only to preservation of the copyright notice and license text. To clone:

```
git clone https://github.com/enclaved/enclaved
```

A separate proprietary product, **enclaved-enclaved**, is distributed by the same publisher under commercial terms. Its internal architecture, code footprint, and feature set are out of scope for this paper.

References

- [1] National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations*, NIST Special Publication 800-53 Rev. 5, 2020.
- [2] National Institute of Standards and Technology. *Security Requirements for Cryptographic Modules*, FIPS 140-3, 2019.
- [3] National Institute of Standards and Technology. *The NIST Cybersecurity Framework (CSF) 2.0*, NIST CSWP 29, 2024.
- [4] National Institute of Standards and Technology. *Protecting Controlled Unclassified Information in Nonfederal Systems and Organizations*, NIST SP 800-171 Rev. 2, 2020.
- [5] International Organization for Standardization. *ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems — Requirements*, 2022.
- [6] American Institute of Certified Public Accountants. *Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy*, 2022.
- [7] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 (General Data Protection Regulation)*, 2016.
- [8] U.S. Department of Health and Human Services. *HIPAA Security Rule, 45 CFR Part 160 and Subparts A and C of Part 164*.

- [9] PCI Security Standards Council. *Payment Card Industry Data Security Standard, v4.0*, 2022.
- [10] D. E. Bell and L. J. LaPadula. *Secure Computer System: Unified Exposition and Multics Interpretation*, MITRE Technical Report MTR-2997, 1976.
- [11] P. Loscocco and S. Smalley. Integrating Flexible Support for Security Policies into the Linux Operating System. In *Proceedings of the FREENIX Track: 2001 USENIX Annual Technical Conference*, 2001.
- [12] S. Haber and W. S. Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2), 1991.
- [13] S. Josefsson and I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*, IETF RFC 8032, 2017.
- [14] S. A. Crosby and D. S. Wallach. Efficient Data Structures for Tamper-Evident Logging. In *18th USENIX Security Symposium*, 2009.
- [15] Anthropic. *Model Context Protocol Specification*, 2024. <https://modelcontextprotocol.io/>.
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [17] L. Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. arXiv:2312.07104, 2023.
- [18] Ollama. <https://github.com/ollama/ollama>.
- [19] LM Studio. <https://lmstudio.ai/>.
- [20] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [21] F. Perez and I. Ribeiro. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527, 2022.
- [22] S. Schulhoff et al. Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition. In *Proceedings of EMNLP*, 2023.
- [23] SPIFFE: Secure Production Identity Framework for Everyone. <https://spiffe.io/>.
- [24] Node.js Foundation. *Node.js node:test module*, 2024. <https://nodejs.org/api/test.html>.
- [25] OpenClaw upstream repository. <https://github.com/openclaw/openclaw>.
- [26] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023.
- [27] International Organization for Standardization. *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*, 2023.

- [28] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*, June 2024.
- [29] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*, 2025 edition. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [30] The MITRE Corporation. *Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)*. <https://atlas.mitre.org/>.
- [31] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel. Extracting Training Data from Large Language Models. In *30th USENIX Security Symposium*, 2021.
- [32] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramer, and K. Lee. *Scalable Extraction of Training Data from (Production) Language Models*. arXiv:2311.17035, 2023.
- [33] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1*, NIST SP 800-218, February 2022.
- [34] Cybersecurity and Infrastructure Security Agency, U.S. National Security Agency, and U.S. Federal Bureau of Investigation. *Engaging with Artificial Intelligence: Joint Guidance for Secure Deployment*, 2024.
- [35] Civil Resolution Tribunal of British Columbia. *Moffatt v. Air Canada*, 2024 BCCRT 149, February 2024.
- [36] K. J. Biba. *Integrity Considerations for Secure Computer Systems*, MITRE Technical Report MTR-3153, 1977.
- [37] D. D. Clark and D. R. Wilson. A Comparison of Commercial and Military Computer Security Policies. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1987.
- [38] D. F. C. Brewer and M. J. Nash. The Chinese Wall Security Policy. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1989.
- [39] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman. Role-Based Access Control Models. *IEEE Computer*, 29(2):38–47, February 1996.
- [40] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162, January 2014.
- [41] J. H. Saltzer and M. D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, September 1975.
- [42] J. Gray. The Transaction Concept: Virtues and Limitations. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, 1981.
- [43] D. Avila and contributors. LibreChat — enhanced ChatGPT clone with multi-provider, plugins, and agents. <https://github.com/danny-avila/LibreChat>.
- [44] Open WebUI Project. Open WebUI — user-friendly self-hosted AI interface (formerly ollama-webui). <https://github.com/open-webui/open-webui>.

- [45] Mintplex Labs. AnythingLLM — the all-in-one AI app for any LLM with full RAG and agent capabilities. <https://github.com/Mintplex-Labs/anything-llm>.
- [46] Jan.ai Project. Jan — bring AI to your desktop. Open-source ChatGPT alternative, runs 100% offline. <https://github.com/janhq/jan>.
- [47] LobeHub. Lobe Chat — open-source modern-design AI chat framework. <https://github.com/lobehub/lobe-chat>.
- [48] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
- [49] H. Chase and the LangChain community. LangChain — building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>.
- [50] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. arXiv:2310.10501, 2023.
- [51] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674, 2023.
- [52] Lakera AI. Lakera Guard — protect your LLM applications from prompt injection, PII leaks, and data poisoning. <https://www.lakera.ai/lakera-guard>.
- [53] A. Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, November 1979.
- [54] L. Lamport, R. Shostak, and M. Pease. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, July 1982.
- [55] National Institute of Standards and Technology. *A Profile for U.S. Federal Cryptographic Key Management Systems*, NIST Special Publication 800-152, October 2015.
- [56] A. Metere (Enclawed LLC). *enclawed — open-source half*. Public repository. <https://github.com/enclawed/enclawed>.
- [57] A. Metere (Enclawed LLC). *Methods for Formal Verification of Agent Skills: Three Layers Toward a Mechanically Checkable Capability-Containment Proof*, 2026. Companion paper to *enclawed*; defines the soundness theorems for the `skill-formal-*.mjs` stack documented in Section 4.3. <https://doi.org/10.5281/zenodo.20100247>.
- [58] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd edition. Wiley-Interscience, 2006. Theorem 2.8.1 (data-processing inequality) gives the per-envelope leakage bound at the refinement-typed dispatch boundary.
- [59] A. Metere (Enclawed LLC). *SKILL.md Manifest Specification*, 2026. Canonical developer-facing reference at <https://docs.openclaw.ai/tools/skill-manifest-spec>.

Module	Tests	Coverage focus
classification.test.mjs	13	lattice arithmetic, format/parse round-trip, immutability
classification-scheme.test.mjs	11	all 5 presets round-trip, custom JSON, validation invariants
dlp-scanner.test.mjs	11	all 3 pattern families, redact correctness
policy.test.mjs	9	enclaved/open defaults, allowlist enforcement, frozen
module-loader.test.mjs	8	8 deny paths and 1 allow path under both flavors
crypto-fips.test.mjs	10	AES-GCM round-trip, AAD round-trip (string + Buffer), AAD mismatch rejection, FIPS gate
zeroize.test.mjs	7	buffer fill, withSecret on success and throw
egress-guard.test.mjs	6	block, allow, lifecycle
module-manifest.test.mjs	6	valid parse, invariant rejection, hash stability
flavor.test.mjs	5	alias parsing, env override, default
audit-log.test.mjs	4	append + verify, tamper, reopen, hash determinism
module-signing.test.mjs	4	Ed25519 round-trip, wrong-key, tampered, malformed
hitl.test.mjs	14	state machine, cooperative checkpoint, pause/resume/stop, approval queue, audit integration
transaction-buffer.test.mjs	18	ramPercent sizing, hash chain, LIFO rollback, partial-failure rollback, eviction, audit integration
paper-conformance.test.mjs	82	§3.2 module-set gating (one subtest per gated cloud module), §5 signing coverage, §3 parallel surfaces, §9 paper-declared test counts
skill-formal-static.test.mjs	17	Method A pattern matches across Python / shell / Node / TS, comment stripping, directory walking, reflective-construct tainting, contained vs non-contained verdicts
skill-formal-types.test.mjs	12	Method B refinement-boundary throws, in / out-of-manifest dispatch, unknown-token rejection, predicate form, JSON-serialisable verdict
skill-formal-bmc.test.mjs	14	Method C reference gate, biconditional violation kinds (executed-without-audit, executed-but-deny, admitted-without-audit), state-space size formula, instanceHash stability
skill-formal-bundle.test.mjs	10	sign / verify round-trip, mismatched-key failure, cache-miss-on-skill-drift, signer-not-authorised rejection, on-disk round-trip, canonicalisation stability, tamper detection
Total OSS unit	261	

Table 7: Unit test inventory.

File	Tests	Adversarial focus
audit-log.pentest.mjs	6	in-place edit, tail truncation (documented limit), record reorder, newline log-injection, <code>__proto__</code> pollution, 100 concurrent appends
signature-forgery.pentest.mjs	7	wrong-key, signerKeyId swap, signature replay across modules, downgrade attack, capability injection, malformed signature, open-flavor warning capture
egress-bypass.pentest.mjs	8	hostname case normalization, Host-header spoofing, embedded credentials, IPv4-in-IPv6, file:/data: schemes, fetch reassign with/without freeze (latter via subprocess), malformed URLs
trust-root-and-scheme.pentest.mjs	10	setTrustRoot after lock, <code>__proto__</code> /constructor pollution, duplicate normalized names, non-contiguous ranks, negative rank, empty levels, non-string canonicalName, missing id, type checks
dlp-evasion.pentest.mjs	8	oversize cap, ReDoS bound, zero-width camouflage, whitespace variants, redact correctness, PEM camouflage gap, null/undefined safety
prompt-injection.pentest.mjs	11	5 spoofed roles, bidi overrides, zero-width camouflage, control chars, code-fence break-out, imperative-override phrases, multi-vector composite, idempotency
code-injection.pentest.mjs	8	shell-metachar manifest fields, JS-eval clearance rejected, function-shaped JSON treated as data, channel id path-traversal denied, <code>__proto__</code> provider id, allowedDirs allowlist, malformed JSON error context, deep-JSON safety
extension-egress-bypass.pentest.mjs	10	raw-socket bypass via <code>net.Socket.connect</code> , <code>net.createConnection</code> (normalized-array form), <code>http.request</code> to literal IPs, subclassed <code>Socket</code> , VPN-CIDR boundary, <code>ipInCidr</code> edges, onDeny audit hook, post-freeze tamper attempts
extension-audit-tamper.pentest.mjs	12	per-field record edits, recordHash forgery propagation, deletion of middle records, re-ordering, 32 concurrent appends, newline log-injection, prototype-pollution payloads, documented tail-truncation gap
extension-net-admission.pentest.mjs	15	unknown capability tokens rejected, empty <code>netAllowedHosts</code> rejected, unsigned in enclaved rejected, signer-not-in-trust-root rejected, post-signing tamper caught (canonical bytes cover <code>verification + netAllowedHosts</code>), <code>net.egress</code> below <code>tested</code> rejected, per-extension <code>Socket</code> guard, biconditional D=S report
Total OSS pen-tests	95	

Table 8: Adversarial test inventory. Each row covers a distinct attack family; cases that document a known limitation are noted as “documented” or “gap” so the deploying organization knows where defense-in-depth is required.